



Школа глубокого обучения ФПИ МФТИ

Домашнее задание. Решение конкурса на kaggle

Это домашнее задание посвящено полноценному решению задачи машинного обучения.

Есть две части этого домашнего задания:

Отправить ваши предсказания в Stepik.

За прохождение определенных порогов будут начисляться баллы. Эта часть оценивается из 5 баллов.

1. $1.00 \geq score \geq 0.84$ --- 5 баллов
2. $0.84 > score \geq 0.77$ --- 4 балла
3. $0.77 > score \geq 0.70$ --- 3 балла
4. $0.70 > score \geq 0.65$ --- 2 балла
5. $0.65 > score \geq 0.6$ --- 1 балл
6. $0.60 > score$ --- 0 баллов

Для этого мы предварительно разделили данные в задании на две части.

- `train.csv`. На этом наборе данных вам необходимо создать и обучить модель. Подробное описание файла есть в ноутбуке.
- `test.csv`. В каждой строчке файла `test.csv` указаны признаки тестовых объектов. Предсказания для этого набора необходимо записать в файл `submission.csv` и сдать в соответствующий шаг на Stepik. Количество попыток ограничено 100 штук.

Отправлять в Stepik вы будете файл с ответами `my_submission.csv`. В этом файле вам необходимо для каждого объекта из датасета предсказать вероятность класса 1. Мы подготовили для вас файл с примером отправки (`submission.csv`). Ваш файл должен быть оформлен таким же образом. В ноутбуке есть пример оформления файла отправки. Для отправки файла с предсказаниями на Stepik есть отдельный шаг.

Сделать полноценный отчет о вашей работе.

Опишите, как вы обработали данные, какие модели попробовали и какие результаты получились (максимум 10 баллов). За каждую выполненную часть будет начислено определенное количество баллов. В этом пункте вам необходимо отправить файл в формате .ipynb на Stepik --- для этого в домашнем задании есть отдельный шаг. Этот пункт оценивается из 10 баллов.

Вторая часть будет проверяться в формате peer-review. Т.е. вашу посылку на Stepik будут проверять 3 других студента, и медианное значение их оценок будет выставлено. Чтобы получить баллы, вам также нужно будет проверить трех других учеников. Это станет доступно после того, как вы сдадите задание сами.

Контекст на Kaggle

Для вашего удобства мы подготовили [контекст на Kaggle](#). Тестирующая система на Kaggle выдает те же баллы, что и Stepik. Мы рекомендуем в качестве основной тестирующей системы использовать именно Kaggle, а затем сдать лучшую посылку на Stepik.

Так вы привыкните работать с Kaggle.

Проблема для российских пользователей Kaggle заключается в том, что подтверждение аккаунта по номеру телефона работает с перебоями или не работает вообще. Без подтвержденного номера телефона вы не сможете совершать отправки в kaggle. Если у вас возникла такая проблема, мы советуем следующие шаги:

- Попробовать зарегистрировать аккаунт с аутентификацией через gmail. В таком случае, аккаунт должен работать без подтверждения номера телефона.
- Если у вас есть не российский номер, используйте его для верификации аккаунта.
- Если на номер телефона не приходит смс для верификации аккаунта, обратитесь в службу поддержки kaggle. Они с переменным успехом умеют подтверждать аккаунты вручную.
- Если все же подтвердить аккаунт и получить доступ к сдаче заданий не удалось, используйте только сдачу в Stepik.

Несколько замечаний по выполнению работы

- Во всех пунктах указания это минимальный набор вещей, которые стоит сделать. Если вы можете сделать какой-то шаг лучше или добавить что-то свое --- дерзайте!
- Пожалуйста, перед сдачей ноутбука убедитесь, что работа чистая и понятная. Это значительно облегчит проверку и повысит ваши ожидаемые баллы.
- Если у вас будут проблемы с решением или хочется совета, то пишите в наш чат в телеграме.

Данные: [train.csv](#), [test.csv](#), [submission.csv](#).

Если ссылки на данные не работают, их можно скачать [на Kaggle](#).

Как проверять?

Ставьте полный балл, если выполнены все рекомендации или сделано что-то более интересное и сложное. За каждый отсутствующий пункт из рекомендации снижайте 1 балл.

Если решение верное, но не удовлетворяет вашим эстетическим предпочтениям, за это баллы снижать не нужно.

Метрика

Перед решением любой задачи важно понимать, как будет оцениваться ваше решение. В данном случае мы используем стандартную для задачи классификации метрику ROC-AUC. Ее можно вычислить, используя только предсказанные вероятности и истинные классы без конкретного порога классификации + она работает даже если классы в данных сильно несбалансированы (примеров одного класса в десятки раз больше примеров другого). Именно поэтому она очень удобна для соревнований.

Посчитать ее легко:

```
from sklearn.metrics import roc_auc_score

y_true = [
    0,
    1,
    1,
    0,
    1
]

y_predictions = [
    0.1,
    0.9,
    0.4,
    0.6,
    0.61
]

roc_auc_score(y_true, y_predictions)

np.float64(0.8333333333333333)
```

Первая часть. Исследование

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import random
```

Загрузка данных (2 балла)

1. Посмотрите на случайные строчки.

- ✓ Посмотрите на случайные строчки.

```
feature_cols = num_cols + cat_cols
target_col = 'Churn'
```

```
data.iloc[[random.randint(0, sh[0]) for _ in range(5)]]
```

	ClientPeriod	MonthlySpending	TotalSpent	Sex	IsSeniorCitizen	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	H
208	58	110.65	6526.65	Female	0	Yes	No	Yes		Yes
450	38	89.10	3342	Female	0	No	No	Yes		No
3897	67	24.90	1680.25	Female	0	Yes	Yes	Yes		Yes
158	40	67.45	2731	Male	0	No	No	Yes		No
5208	20	20.05	406.05	Female	0	Yes	Yes	Yes		No

- data.isna() или data.info() и, если нужно, замените их на что-то. Будет хорошо,

если вы построите табличку с количеством nan в каждой колонке.

```
data.isna().sum()
```

```
ClientPeriod      0
MonthlySpending   0
TotalSpent        0
Sex               0
IsSeniorCitizen   0
HasPartner        0
HasChild          0
HasPhoneService   0
HasMultiplePhoneNumbers 0
HasInternetService 0
HasOnlineSecurityService 0
HasOnlineBackup   0
HasDeviceProtection 0
HasTechSupportAccess 0
HasOnlineTV       0
HasMovieSubscription 0
HasContractPhone  0
IsBillingPaperless 0
PaymentMethod     0
Churn             0
dtype: int64
```

Анализ данных (3 балла)

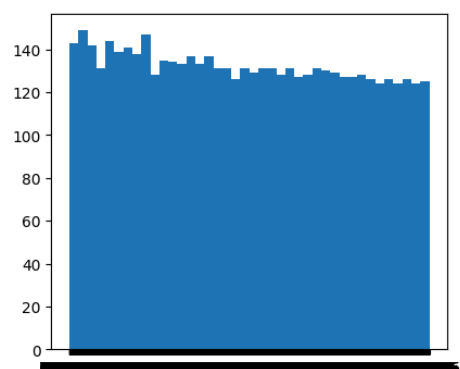
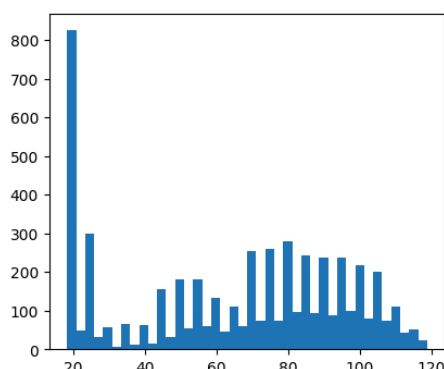
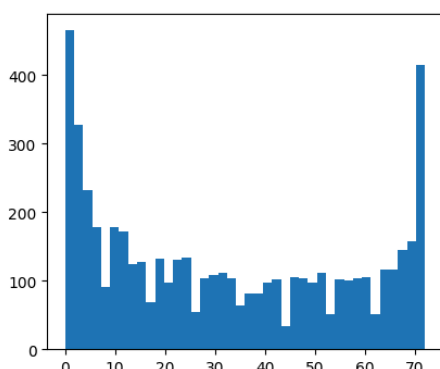
1. Для численных признаков постройте гистограмму (*plt.hist(...)*) или boxplot (*plt.boxplot(...)*). Для категориальных посчитайте количество каждого значения для каждого признака. Для каждой колонки надо сделать *data.value_counts()* и построить bar диаграммы *plt.bar(...)* или круговые диаграммы *plt.pie(...)* (хорошо, если вы сможете это сделать на одном графике с помощью *plt.subplots(...)*).
2. Посмотрите на распределение целевой переменной и скажите, являются ли классы несбалансированными.
3. (Если будет желание) Поиграйте с разными библиотеками для визуализации - *sns*, *pandas_visual_analysis*, etc.

Второй пункт очень важен, потому что существуют задачи классификации с несбалансированными классами. Например, это может значить, что в датасете намного больше примеров 0 класса. В таких случаях нужно 1) не использовать accuracy как метрику 2) использовать методы борьбы с imbalanced dataset (обычно если датасет сильно несбалансирован, т.е. класса 1 в 20 раз меньше класса 0).

Для численных признаков постройте гистограмму (*plt.hist(...)*) или boxplot (*plt.boxplot(...)*). Для категориальных посчитайте количество каждого значения для каждого признака. Для каждой колонки надо сделать *data.value_counts()* и построить bar диаграммы *plt.bar(...)* или круговые диаграммы *plt.pie(...)* (хорошо, если вы сможете это сделать на одном графике с помощью *plt.subplots(...)*).

```
# YOUR CODE
a, axis = plt.subplots(1, 3, figsize = (16,4))

for i,e in enumerate(num_cols):
    axis[i].hist(data[num_cols[i]], bins = 40)
```

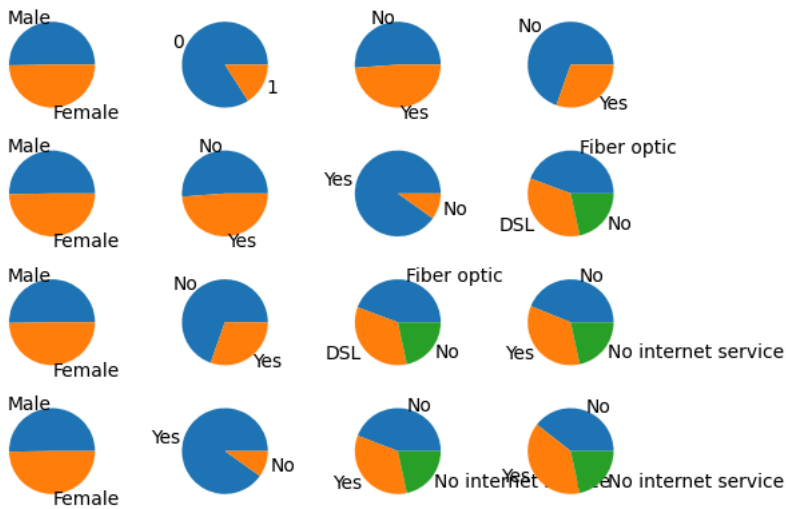


```
# YOUR CODE
aa, axiss = plt.subplots(4, 4)
```

```

for i in range(4):
    for t in range(4):
        temp = data[cat_cols[t*i+t]].value_counts()
        axiss[i][t].pie(temp, labels=temp.index)

```



Посмотрите на распределение целевой переменной и скажите, являются ли классы несбалансированными.

```

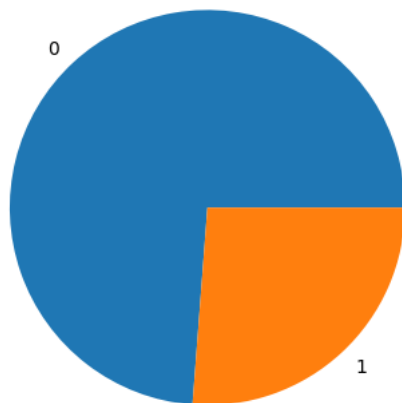
t = data[target_col].value_counts()
plt.pie(t, labels = t.index)

```

```

([<matplotlib.patches.Wedge at 0x78d790116540>,
 <matplotlib.patches.Wedge at 0x78d7885926c0>],
 [Text(-0.7478930544993437, 0.8066324931662757, '0'),
  Text(0.7478931250173959, -0.8066324277833826, '1')])

```



Да, классы являются несбалансированными

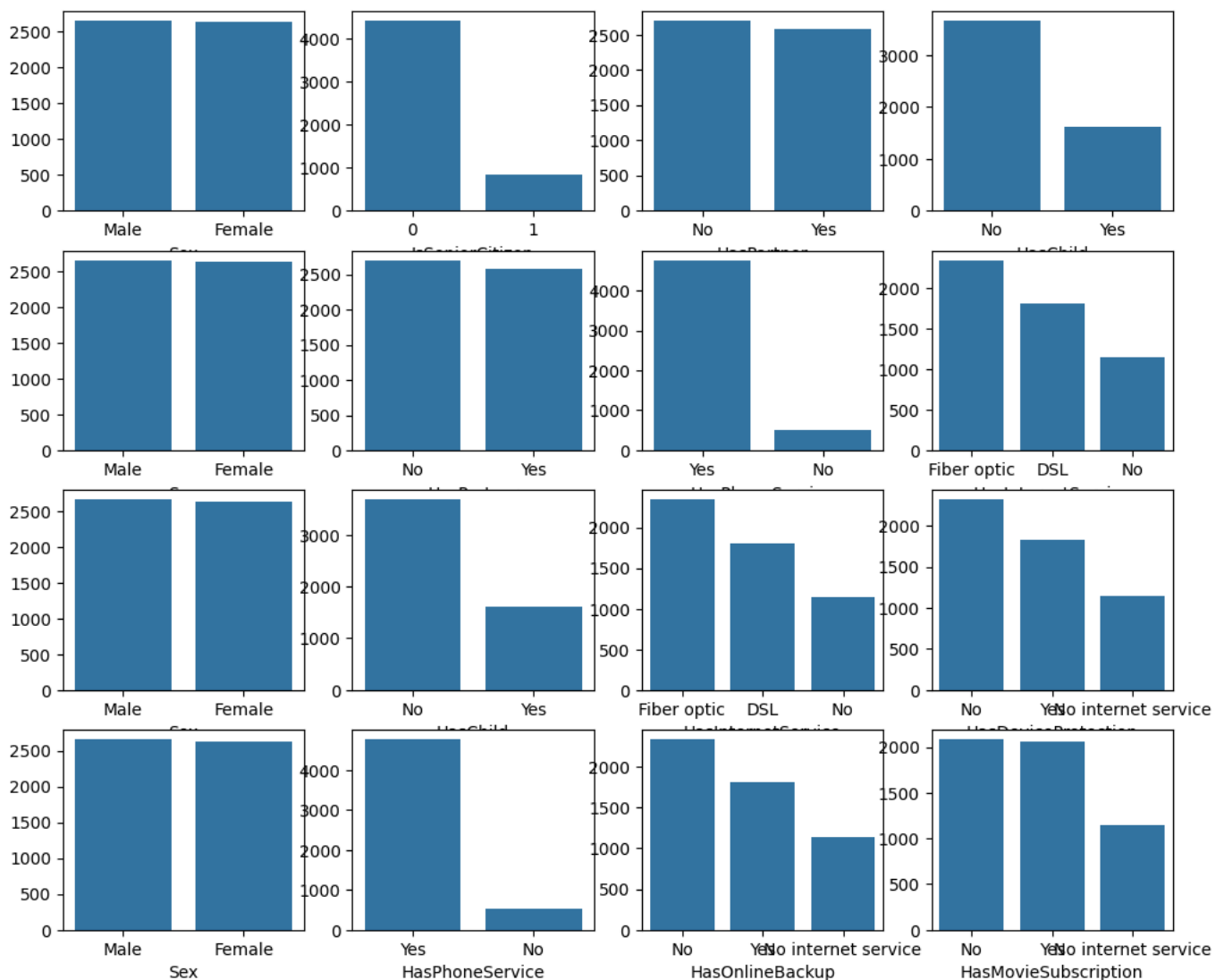
Если будет желание) Поиграйте с разными библиотеками для визуализации - sns, pandas_visual_analysis, etc.

```

import seaborn as sns
aa, axiss = plt.subplots(4, 4, figsize = (12,10))

for i in range(4):
    for t in range(4):
        temp = data[cat_cols[t*i+t]].value_counts()
        sns.barplot(x=temp.index, y=temp.values, ax=axiss[i][t])

```



(Дополнительно) Если вы нашли какие-то ошибки в данных или выбросы, то можете их убрать. Тут можно поэкспериментировать с обработкой данных как угодно, но не за баллы.

Применение линейных моделей (3 балла)

1. Обработайте данные для того, чтобы к ним можно было применить LogisticRegression. Т.е. отнормируйте числовые признаки, а категориальные закодируйте с помощью one-hot-encoding'a.
2. С помощью кроссвалидации или разделения на train/valid выборку протестируйте разные значения гиперпараметра C и выберите лучший (можно тестировать C=100, 10, 1, 0.1, 0.01, 0.001) по метрике ROC-AUC.

Если вы разделяете на train/valid, то используйте LogisticRegressionCV. Он сам при вызове .fit() подберет параметр C. (не забудьте передать scroing='roc_auc', чтобы при кроссвалидации сравнивались значения этой метрики, и refit=True, чтобы при потом модель обучилась на всем датасете с лучшим параметром C).

(более сложный вариант) Если вы будете использовать кроссвалидацию, то преобразования данных и LogisticRegression нужно соединить в один Pipeline с помощью make_pipeline, как это делалось во втором семинаре. Потом pipeline надо передать в GridSearchCV. Для one-hot-encoding'a можно использовать комбинацию LabelEncoder + OneHotEncoder (сначала превращаем строки в числа, а потом числа превращаем в one-hot вектора.)

Обработайте данные для того, чтобы к ним можно было применить

- ✓ LogisticRegression. Т.е. отнормируйте числовые признаки, а категориальные закодируйте с помощью one-hot-encoding'a.

```
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, RobustScaler, LabelEncoder, OneHotEncoder
from sklearn.pipeline import make_pipeline
```

```
X_train, X_test, y_train, y_test = train_test_split(data[feature_cols], data[target_col], random_state = 41, shuffle = True)
```

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

```
a = [float(i) for i in X_train['TotalSpent'] if i != ' ']  
mn = round(np.mean(a), 3)  
X_train['TotalSpent'] = X_train['TotalSpent'].replace(' ', mn)  
X_test['TotalSpent'] = X_test['TotalSpent'].replace(' ', mn)
```

```
a = StandardScaler()  
X_train[num_cols] = a.fit_transform(X_train[num_cols])  
X_test[num_cols] = a.transform(X_test[num_cols])  
  
onehot = OneHotEncoder(sparse_output = False)  
onehot.fit(X_train[cat_cols])  
names = onehot.get_feature_names_out(cat_cols)  
X_tr = pd.DataFrame(onehot.transform(X_train[cat_cols]), columns=names, index=X_train.index)  
X_ts = pd.DataFrame(onehot.transform(X_test[cat_cols]), columns=names, index=X_test.index)
```

```
X_train = X_train.drop(cat_cols, axis = 1)  
X_test = X_test.drop(cat_cols, axis = 1)  
X_train = pd.concat([X_train, X_tr], axis = 1)  
X_test = pd.concat([X_test, X_ts], axis = 1)
```

С помощью кроссвалидации или разделения на train/valid выборку

- ✎ протестируйте разные значения гиперпараметра C и выберите лучший (можно тестировать C=100, 10, 1, 0.1, 0.01, 0.001) по метрике ROC-AUC.

```
from sklearn.metrics import roc_auc_score  
  
cs = [100, 10, 1, 0.1, 0.01, 0.001]  
roca = {}  
for i in cs:  
    logregl = LogisticRegression(C = i, random_state = 41, max_iter = 1000)  
    ft = logregl.fit(X_train, y_train)  
  
    y_predl = logregl.predict_proba(X_test)[: , 1]  
    roc_aucl = roc_auc_score(y_test, y_predl)  
    roca[i] = roc_aucl
```

roca

```
{100: np.float64(0.8661391938692802),  
10: np.float64(0.8662282317687227),  
1: np.float64(0.8658812909881365),  
0.1: np.float64(0.8651843391545697),  
0.01: np.float64(0.8630044457544273),  
0.001: np.float64(0.8583314911698966)}
```

- ✎ Максимальное значение roc_auc получилось при C = 10: roc_auc = 0.8662

```
logreg = LogisticRegressionCV(cv=5, scoring='roc_auc', random_state=41, max_iter=1000, class_weight='balanced')  
  
f = logreg.fit(X_train, y_train)# Cs=[100, 10, 1, 0.1, 0.01, 0.001], class_weight='balanced'
```

```
from sklearn.metrics import roc_auc_score  
  
y_pred = logreg.predict_proba(X_test)[: , 1]  
roc_auc = roc_auc_score(y_test, y_pred)  
roc_auc  
  
np.float64(0.8663111291233757)
```

Выпишите какое лучшее качество и с какими параметрами вам удалось получить

- ✎ Лучшее качество получилось с C = 10, оно равняется 0.86622

С кросс-валидацией удалось получить roc_auc = 0.8663

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

Применение градиентного бустинга (2 балла)

Если вы хотите получить баллы за точный ответ, то стоит попробовать градиентный бустинг. Часто градиентный бустинг с параметрами по умолчанию даст вам 80% результата за 0% усилий.

Мы будем использовать catboost, поэтому нам не надо кодировать категориальные признаки. Catboost сделает это сам (в .fit() надо передать cat_features=cat_cols). А численные признаки нормировать для моделей, основанных на деревьях не нужно.

1. Разделите выборку на train/valid. Протестируйте catboost со стандартными параметрами.
2. Протестируйте разные значения параметров количества деревьев и learning_rate'a и выберите лучшую по метрике ROC-AUC комбинацию.

(Дополнительно) Есть некоторые сложности с тем, чтобы использовать CatBoostClassifier вместе с GridSearchCV, поэтому мы не просим использовать кроссвалидацию. Но можете попробовать)

Разделите выборку на train/valid. Протестируйте catboost со стандартными параметрами.

```
X_trainb, X_testb, y_trainb, y_testb = train_test_split(data[feature_cols], data[target_col], random_state = 41, shuffle = True)
```

```
a = [float(i) for i in X_trainb['TotalSpent'] if i != ' ']  
mn = round(np.mean(a), 3)  
X_trainb['TotalSpent'] = X_trainb['TotalSpent'].replace(' ', mn)  
X_testb['TotalSpent'] = X_testb['TotalSpent'].replace(' ', mn)
```

```
from catboost import CatBoostClassifier
```

```
boost = CatBoostClassifier(cat_features = cat_cols)  
boost.fit(X_trainb, y_trainb, verbose=False)  
y_predb = boost.predict_proba(X_testb)[:, 1]  
roc_aucb = roc_auc_score(y_testb, y_predb)  
roc_aucb
```

```
np.float64(0.8618853314666076)
```

```
from tqdm import tqdm
```

Протестируйте разные значения параметров количества деревьев и learning_rate'a и выберите лучшую по метрике ROC-AUC комбинацию.

```
score = {}  
lr = [0.3, 0.1, 0.05, 0.03, 0.01]  
trees_c = [100, 500, 1000, 2000]  
best = 0.  
for l in tqdm(lr):  
    score[l] = {}  
    for t in trees_c:  
        boost = CatBoostClassifier(cat_features = cat_cols, learning_rate = l, iterations = t, auto_class_weights='Balanced' )  
        boost.fit(X_trainb, y_trainb, verbose=False)  
        y_predb = boost.predict_proba(X_testb)[:, 1]  
        roc_aucb = roc_auc_score(y_testb, y_predb)  
  
        best = max(best, roc_aucb)  
        score[l][t] = roc_aucb
```

```
100%|██████████| 5/5 [02:06<00:00, 25.24s/it]
```

```
print(best)  
score
```

```
0.8661883182275931  
{0.3: {100: np.float64(0.8599126814530986),  
      500: np.float64(0.8355500700022105),  
      1000: np.float64(0.831511126667158),  
      2000: np.float64(0.8285912976199249)},  
 0.1: {100: np.float64(0.8661883182275931),  
      500: np.float64(0.8501768476899271),  
      1000: np.float64(0.8462515044334734),
```

```
2000: np.float64(0.8402706138088571)),
0.05: {100: np.float64(0.8638764031144843),
500: np.float64(0.8564125709232924),
1000: np.float64(0.8520220813990618),
2000: np.float64(0.8441360867536167)},
0.03: {100: np.float64(0.8612774175324835),
500: np.float64(0.8641204897698523),
1000: np.float64(0.8591220863114974),
2000: np.float64(0.8528280279026356)},
0.01: {100: np.float64(0.8500156583892122),
500: np.float64(0.8645963819910103),
1000: np.float64(0.8656494854223465),
2000: np.float64(0.864086716773512)}}
```

```
boost = CatBoostClassifier(cat_features = cat_cols, learning_rate = 0.101, iterations = 100, auto_class_weights='Balanced', l2_leaf_reg =
boost.fit(X_trainb, y_trainb, verbose=False)
y_predb = boost.predict_proba(X_testb)[: , 1]
roc_aucb = roc_auc_score(y_testb, y_predb)
```

roc_aucb

np.float64(0.8679076707685505)

Выпишите, какое лучшее качество и с какими параметрами вам удалось получить

Максимальное значение при изменении стандартных параметров получилось lr = 0.1, iterations = 100; roc_auc = 0.86618

Максимальное значение получилось при lr= 0.101, trees = 100, балансе классов, с l2 регуляризацией 1.5 и увеличенным количество интервалов border_count = 300

roc_auc = 0.8679

Предсказания

```
best_model = boost
```

```
X_test[num_cols].dtypes
```

```
ClientPeriod      int64
MonthlySpending   float64
TotalSpent         float64
dtype: object
```

```
X_test = pd.read_csv('./test.csv')
submission = pd.read_csv('./submission.csv')
```

```
submission['Churn'] = best_model.predict_proba(X_test)[: , 1]
submission.to_csv('./my_submission1.csv')
```

Лучшее решение отправить в Stepik.

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.